

BillieBot Cameo/MSOSA Model Implementation Report

System-of-systems model structure, SysML diagram implementation guide, activity-diagram behavior, and requirements satisfaction traceability

Field	Value
System	BillieBot - AI dog companion / dog behavior robot MVP
Modeling Environment	Cameo Systems Modeler / Magic Systems of Systems Architect (MSOSA) using SysML-style artifacts
Primary Objective	Provide a buildable model architecture for MVP navigation, detection, classification, logging, and daily summary generation
MVP Boundary	No Behavior AI, reinforcement learning, autonomous social engagement, treat dispensing, or action-policy learning
Future-Proofing	Reserve inactive interfaces and blocks for Behavior AI and learning-loop integration

This report is intended to be used as a practical model-building checklist. It names the packages, blocks, requirements, activities, flows, and trace links that should be created in Cameo/MSOSA. The visual diagrams are not merely illustrative; each one corresponds to a diagram or matrix that should exist in the model.

1. Executive Summary

BillieBot should be represented as a small system-of-systems model with a strict MVP boundary. The executable MVP behavior is a deterministic robotic sensing mission: navigate the apartment, search for Billie, detect visual and audio evidence, classify Billie state, log evidence, and generate a daily report. Future Behavior AI should be present only as reserved, inactive architectural space.

The Cameo model should emphasize traceability. Every MVP requirement should be connected to one or more functional activities, logical blocks, physical components, software nodes, and verification test cases. This lets the model remain useful during implementation instead of becoming a decorative architecture diagram.

Recommended model layers:

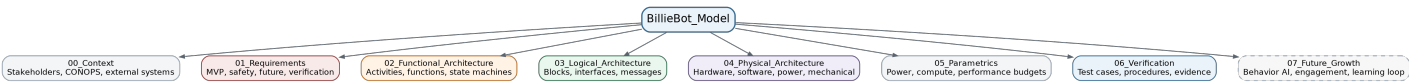


Figure 1. Recommended BillieBot package hierarchy for a Cameo/MSOSA model.

Layer	Purpose	Primary Cameo artifacts
Context	Defines the operational boundary around BillieBot, Billie, owner, apartment, host computer, and WiFi router.	Context BDD, use case/operational concept diagram, external block definitions
Requirements	Captures the MVP requirements, safety requirements, and future reserved requirements.	Requirements diagrams, requirement tables, traceability matrix
Functional	Defines what BillieBot must do independent of hardware.	Activity diagrams, functional BDD, allocations
Logical	Defines system subsystems and software responsibilities.	BDD, IBD, interface blocks, item flows
Physical	Allocates logical functions to Jetson, sensors, Arduino, mobile base, power, and network components.	Physical BDD/IBD, allocation tables
Verification	Defines tests that verify requirements and produce evidence.	Test case diagrams, verification matrix, evidence references
Future Growth	Preserves Behavior AI integration points without including them in the MVP behavior.	Inactive blocks, future message schemas, constraints

2. Cameo/MSOSA Model Setup

Create a single project named **BillieBot_Model**. Within it, create packages matching the hierarchy in Figure 1. Keep the package names prefixed with numbers so Cameo browser ordering remains stable. This is helpful when the model grows to include requirements, test cases, and future Behavior AI.

Recommended model conventions

Convention	Recommendation
Element names	Use PascalCase for blocks and activities, e.g., BillieBot, NavigationSubsystem, FindBillieActivity.
Requirement IDs	Use stable IDs such as REQ-MVP-001, REQ-SAFE-003, REQ-FUT-001. Never encode page or diagram numbers in IDs.
Diagrams	Prefix with diagram type and number, e.g., BDD-01 System Context, ACT-02 Find Billie, IBD-03 MVP Data Flow.
Inactive future elements	Place future behavior blocks in 07_Future_Growth and mark diagrams/notes as "reserved - not active in MVP".
Traceability	Use satisfy for block-to-requirement, refine for activity-to-requirement, verify for test-case-to-requirement, and allocate for function-to-physical/software implementation.
Interfaces	Use ports and item flows for ROS topics, hardware buses, power rails, and future AI context/action messages.

Minimum SysML stereotypes / element types to use

Need	Cameo/SysML element
System, subsystem, sensor, computer, software node	Block
Whole-part subsystem decomposition	Composition association or part property
Typed boundary interaction	Proxy port or full port
Data, energy, command, and status crossing interfaces	Item flow, flow property, interface block
Behavior sequence	Activity with actions, decisions, object nodes, and control/object flows
MVP shall statements	Requirement
Proof that design addresses requirement	Satisfy, refine, allocate, verify relationships
Test procedure	TestCase stereotype or test case block/activity

Implementation note

For early model construction, prioritize clear diagram semantics over exact Cameo ornamentation. The essential goal is that the model browser contains reusable elements and that diagrams display views of those elements, not duplicate text boxes. Build blocks, requirements, activities, and interfaces once, then drag those same elements onto multiple diagrams as needed.

3. System Context Model

The context model defines what BillieBot is responsible for and what it merely interacts with. In Cameo, create a **BDD-01 System Context** diagram in package **00_Context**. BillieBot is the system of interest. Billie, the owner, the apartment, the host computer, the robot-net WiFi router, and optional cloud services are external or enabling systems.

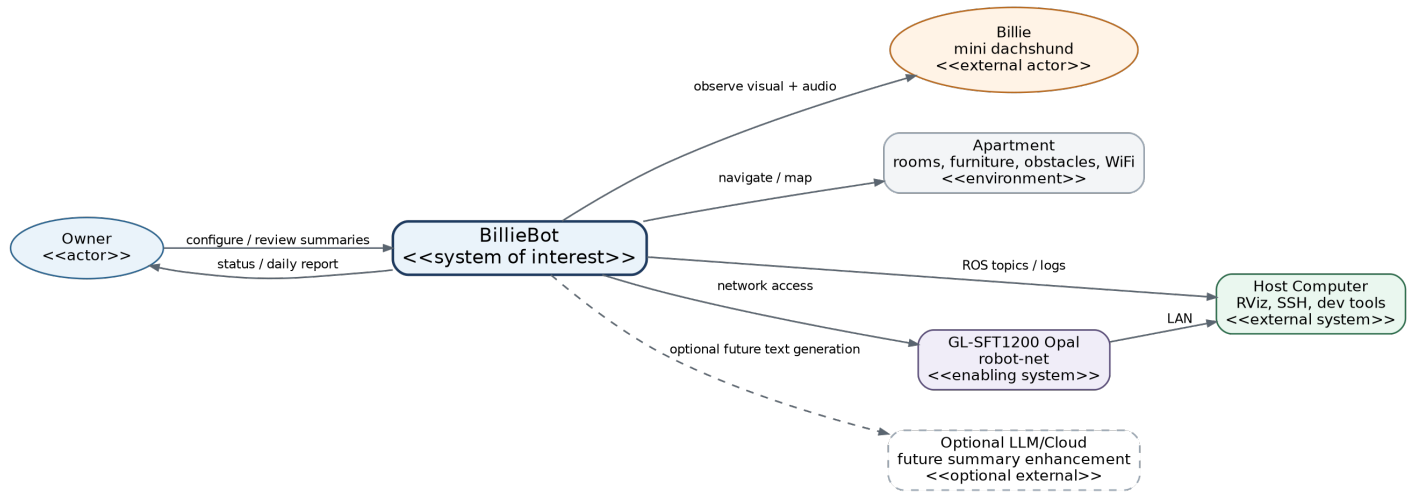


Figure 2. Context BDD showing BillieBot boundary and external systems.

Element	Type	Modeling instruction
BillieBot	Block	Mark as system of interest. All MVP subsystems should compose into this block.
Billie	External actor/block	Represents the dog being observed. Do not model Billie as a controllable subsystem.
Owner	Actor/block	Configures missions, reviews summaries, and can stop the robot.
Apartment	Environment block	Owns rooms, obstacles, WiFi coverage, and safety constraints.
HostComputer	External system	Development, RViz, SSH, logging access, test execution.
WiFiRouter_Opal	Enabling system	Provides local communication network between robot and host computer.
OptionalLLMCloud	Optional external system	Future summary enhancement only. Not required for MVP operation.

Requirements satisfied by this diagram

The context diagram supports REQ-MVP-009 and REQ-MVP-010 by showing that future social engagement or cloud-enabled intelligence is outside the MVP boundary. It also supports all verification planning by making clear which external systems are needed for tests: the apartment, owner/host computer, WiFi router, and Billie or Billie-like test object.

4. Requirements Model

Create a requirements package with separate diagrams for MVP requirements, safety requirements, future reserved requirements, and verification requirements. The requirements hierarchy should be the controlling source for scope. Do not allow future Behavior AI requirements to be traced into executable MVP activities.

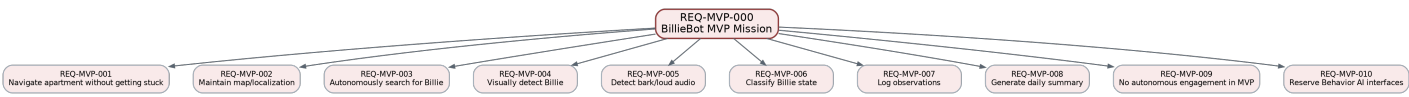


Figure 3. MVP requirements hierarchy to create in a Cameo Requirements Diagram.

Core MVP requirements

ID	Requirement text	Primary verification
REQ-MVP-001	BillieBot shall autonomously navigate the apartment without getting stuck.	30-minute navigation/search run without manual rescue.
REQ-MVP-002	BillieBot shall maintain a map or localization representation of the apartment.	SLAM map or AMCL localization topics available and stable.
REQ-MVP-003	BillieBot shall autonomously search for Billie.	Waypoint search route completes or reports structured failure.
REQ-MVP-004	BillieBot shall visually detect Billie using a visible or IR camera.	Dog/Billie detections published on /billie/detections.
REQ-MVP-005	BillieBot shall auditorily detect when Billie barks or makes other loud noises.	AudioEvent messages published for bark/loud-noise tests.
REQ-MVP-006	BillieBot shall classify and publish Billie state observations.	BillieStateObservation published with label and confidence.
REQ-MVP-007	BillieBot shall log observations with timestamp and evidence source.	SQLite rows created for state, visual, and audio events.
REQ-MVP-008	BillieBot shall generate a daily activity summary.	Summary text/Markdown file and daily_summaries row created.
REQ-MVP-009	BillieBot shall not execute Behavior AI engagement actions in the MVP.	No policy/treat/speech action nodes active in MVP launch path.
REQ-MVP-010	BillieBot shall provide reserved interfaces for future Behavior AI integration.	Future interfaces are modeled and documented but inactive.

5. Logical Architecture BDD

Create **BDD-02 Logical Architecture** in package **03_Logical_Architecture**. This diagram decomposes BillieBot into the logical subsystems that will later be allocated to software packages, hardware devices, and ROS 2 nodes. The logical model should remain mostly independent of vendor hardware details.

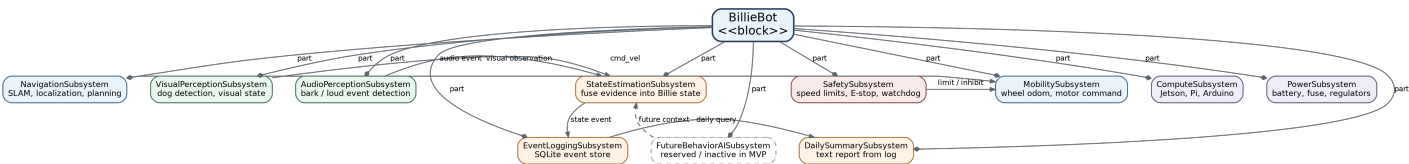


Figure 4. Logical BDD for BillieBot MVP subsystems and future reserved Behavior AI subsystem.

Key modeling choices

Subsystem	MVP responsibility	Requirements primarily satisfied
NavigationSubsystem	SLAM, localization, waypoint navigation, obstacle avoidance, recovery.	REQ-MVP-001, REQ-MVP-002, REQ-MVP-003
MobilitySubsystem	Differential-drive actuation, odometry, motor command interface.	REQ-MVP-001
VisualPerceptionSubsystem	Camera ingestion, dog/Billie detection, visual state cues.	REQ-MVP-004, REQ-MVP-006
AudioPerceptionSubsystem	Audio loudness/bark/loud-noise event detection.	REQ-MVP-005, REQ-MVP-006
StateEstimationSubsystem	Rule-based evidence fusion into Billie state labels.	REQ-MVP-006
EventLoggingSubsystem	Structured event storage with timestamps and confidence.	REQ-MVP-007
DailySummarySubsystem	Deterministic daily report generation from logs.	REQ-MVP-008
SafetySubsystem	Speed limiting, watchdog, software E-stop, safe stop modes.	REQ-MVP-001, REQ-MVP-009, safety requirements
FutureBehaviorAISubsystem	Reserved interfaces only; inactive in MVP.	REQ-MVP-010

How this satisfies requirements

The logical architecture satisfies requirements by assigning each required capability to a named subsystem. This is the first level of design accountability: a requirement without a satisfying subsystem is unimplemented; a subsystem without a requirement is either support infrastructure or scope creep. The FutureBehaviorAISubsystem is intentionally present but inactive, which satisfies future-readiness without violating REQ-MVP-009.

6. MVP Internal Block Diagram

Create **IBD-01 MVP Data and Control Flow** under the logical or physical architecture package. The IBD should show typed ports and item flows for sensor data, ROS topics, velocity commands, odometry, and logging outputs. The diagram below can be reproduced with ports on each subsystem block.

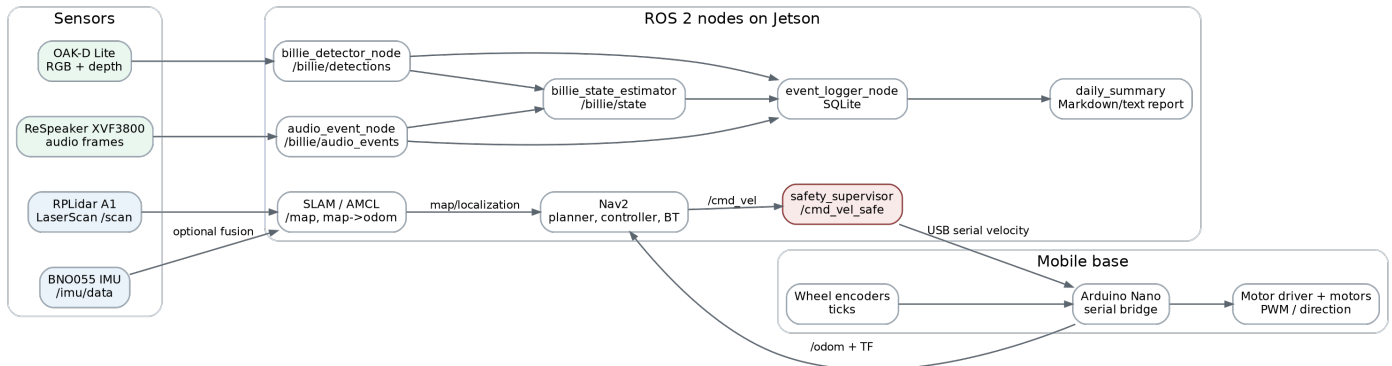


Figure 5. MVP data/control flow IBD showing sensors, ROS nodes, mobile base, and logger/summary path.

Recommended interface blocks / item flows

Interface / flow	Source	Destination	Signal carried
LaserScanFlow	RPLidarA1	SLAM/Localization/Nav2	2D range scan /scan
ImageDepthFlow	OAKDLite	VisualPerceptionSubsystem	RGB image, depth image, camera info
AudioFrameFlow	ReSpeakerXVF3800	AudioPerceptionSubsystem	audio frames or audio features
VelocityCommandFlow	Nav2/SafetySubsystem	MotorControlBridge	/cmd_vel or /cmd_vel_safe
OdometryFlow	Arduino/EncoderOdom	NavigationSubsystem	/odom and odom->base_link TF
BillieDetectionFlow	VisualPerceptionSubsystem	StateEstimationSubsystem	BillieDetectionArray
AudioEventFlow	AudioPerceptionSubsystem	StateEstimationSubsystem	AudioEvent
StateObservationFlow	StateEstimationSubsystem	EventLoggingSubsystem	BillieStateObservation
DailySummaryFlow	DailySummarySubsystem	Owner/HostComputer	Markdown/text daily report

7. Activity Diagram Architecture

Activity diagrams should describe BillieBot behavior at three levels: mission orchestration, capability-specific behavior, and support/safety behavior. In Cameo, build each activity with control flows, object flows, decision nodes with guard labels, and activity partitions. Use call behavior actions so lower-level activities are reusable.

Recommended activity package contents

Activity ID	Activity name	Purpose
ACT-01	ExecuteMvpAutonomousMission	Top-level mission behavior from launch through search, perception, logging, and daily summary.
ACT-02	FindBillie	Waypoint search, visual scan, detection threshold, track, publish detection.
ACT-03	DetectAudioEvent	Continuous audio monitoring, loudness threshold, bark/loud-noise event publication.
ACT-04	ClassifyAndLogBillieState	Fuse visual/audio/pose evidence, classify state, write database rows.
ACT-05	GenerateDailySummary	Query event database, aggregate, compose report, save output.
ACT-06	EnforceSafeMotion	Clamp velocity and stop motion under E-stop, timeout, obstacle, or fault condition.

Activity partition guidance

Partition	Actions placed in partition
MissionManager	Launch stack, health check, select waypoint, mission loop, timeout/low-battery decisions.
NavigationSubsystem	Load map, localize, navigate to waypoint, recover from navigation failure.
VisualPerceptionSubsystem	Acquire image, run detector, track detection, publish BillieDetection.
AudioPerceptionSubsystem	Read audio frame, compute features, classify/publish audio events.
StateEstimationSubsystem	Associate observations, classify state, publish BillieStateObservation.
EventLoggingSubsystem	Write SQLite rows, publish logging status.
DailySummarySubsystem	Aggregate daily events and compose report.
SafetySubsystem	Clamp/stop velocities, set safe_stop mode, issue watchdog stop.

8. ACT-01 ExecuteMvpAutonomousMission

This is the primary behavior diagram for the MVP. It should be built as an activity with call behavior actions to the lower-level activities: FindBillie, DetectAudioEvent, ClassifyAndLogBillieState, GenerateDailySummary, and EnforceSafeMotion. In the diagram, audio monitoring may be shown as continuous/asynchronous behavior, but the mission-level control loop should remain deterministic.

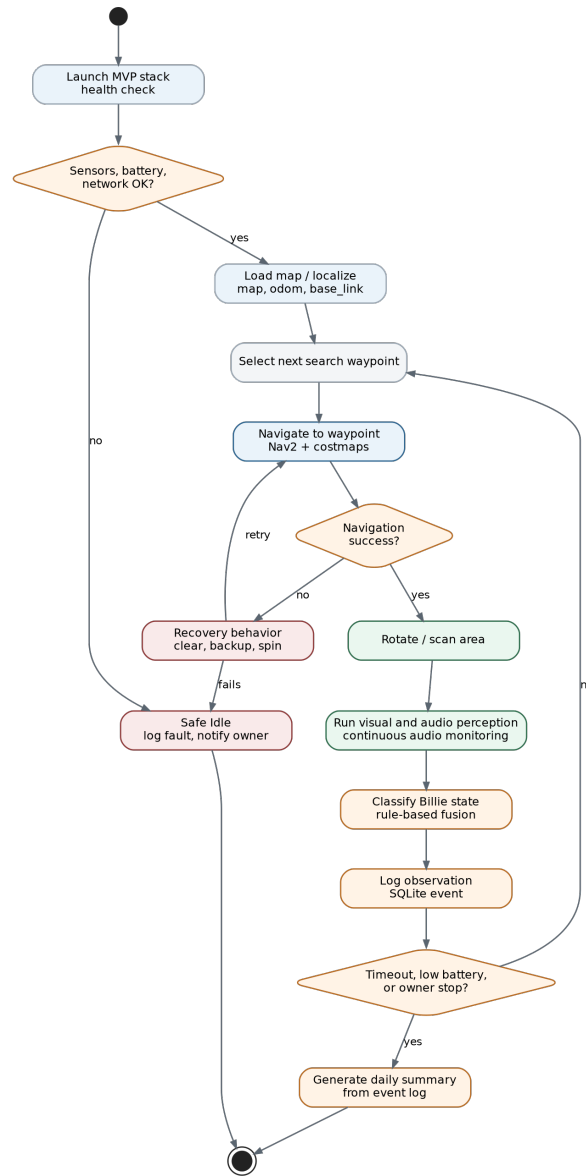


Figure 6. ACT-01 ExecuteMvpAutonomousMission: top-level MVP activity.

Cameo construction notes

Activity element	Cameo implementation
Launch MVP stack	Opaque action or call behavior action referencing software launch/bring-up behavior.
Health check decision	Decision node with guards [yes] and [no]. Object inputs include BatteryStatus, SensorHealth, NetworkStatus.
Navigate to waypoint	Call behavior action to Nav2 waypoint navigation behavior.
Recovery behavior	Call behavior action with outcomes [retry] and [fails].
Run visual and audio perception	Can be decomposed into parallel activity region or call behavior actions to ACT-02 and ACT-03.
Classify and log observation	Call behavior action to ACT-04.
Generate daily summary	Call behavior action to ACT-05 at mission end or scheduled trigger.

Requirements satisfaction

Requirement	How ACT-01 addresses it
REQ-MVP-001	Contains navigation loop, recovery behavior, and safe idle path.
REQ-MVP-002	Requires map/localization before the search loop begins.
REQ-MVP-003	Controls deterministic search-waypoint loop.
REQ-MVP-004/005	Invokes visual and audio perception activities.
REQ-MVP-006/007	Invokes classification and logging after perception.
REQ-MVP-008	Invokes summary generation at mission end or daily trigger.
REQ-MVP-009	No action path includes speech, treat dispensing, or autonomous social engagement.

9. ACT-02 FindBillie

This diagram is the core search-and-detect behavior. In Cameo, it should live under the VisualPerception or Functional_Architecture package, with allocations to NavigationSubsystem and VisualPerceptionSubsystem. The output is a BillieDetection or not_seen/search-failed result, not an engagement action.

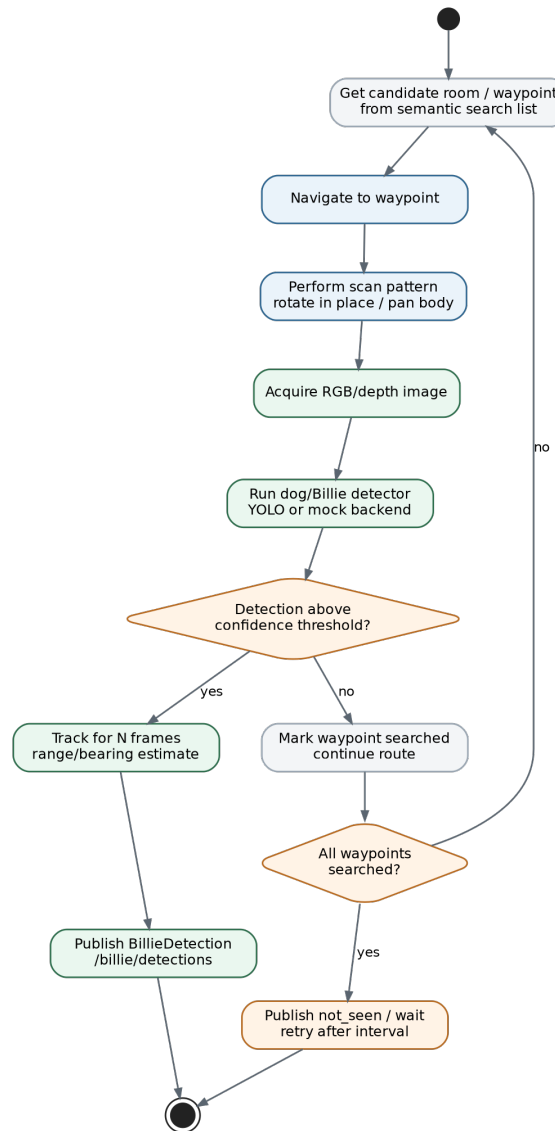


Figure 7. ACT-02 FindBillie: waypoint search and visual detection behavior.

Inputs, outputs, and guard conditions

Model item	Recommended definition
Input object nodes	SearchWaypointList, RobotPose, RGBImage, DepthImage, DetectorConfig.
Output object nodes	BillieDetectionArray, BillieNotFound event, waypoint search status.
Decision guard	[confidence >= detection_threshold] from detector output.
Track for N frames	Use as a call behavior or action to reduce false positives before publishing detection.
All waypoints searched?	Decision node driven by search waypoint state; output not_seen if no candidate found.

Requirements satisfaction

ACT-02 directly refines REQ-MVP-003 and REQ-MVP-004. It also contributes to REQ-MVP-006 and REQ-MVP-007 because detections become evidence for state classification and logging. It does not satisfy engagement behavior because the output

stops at observation, not action selection.

10. ACT-03 DetectAudioEvent

This activity should model continuous listening as a repeated frame-processing loop. The MVP can be threshold-based: compute loudness, apply duration/cooldown rules, classify as bark/loud_noise/unknown, then publish an AudioEvent. A later learned bark classifier can replace the classifier action without changing the activity boundary.

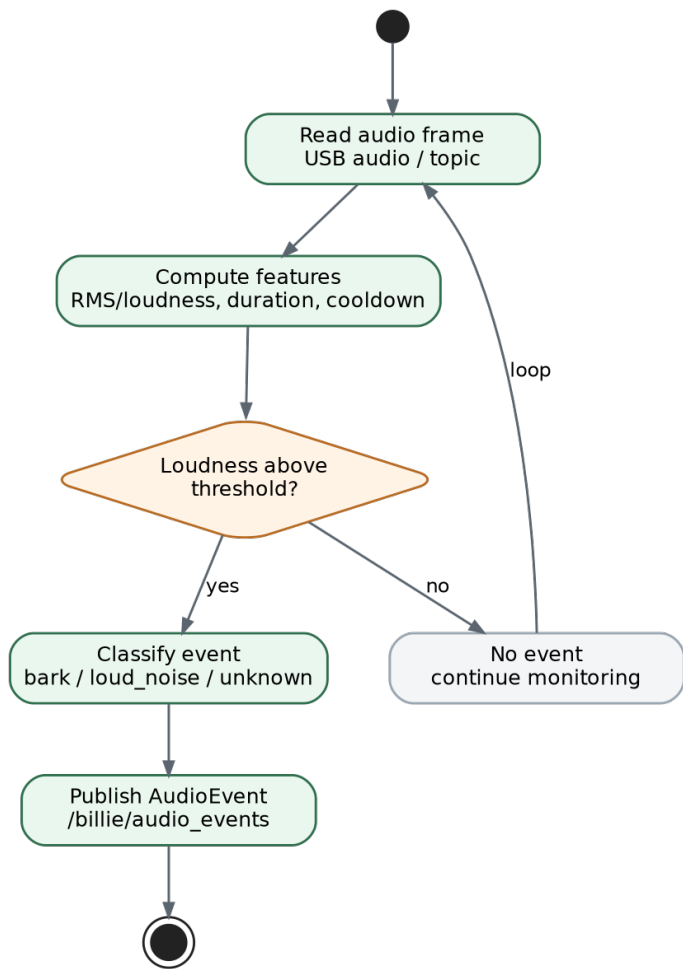


Figure 8. ACT-03 DetectAudioEvent: audio monitoring and event publication.

Requirement	Satisfaction mechanism
REQ-MVP-005	AudioEvent is published when bark or loud-noise conditions are met.
REQ-MVP-006	AudioEvent provides evidence for barking/loud_noise BillieStateObservation.
REQ-MVP-007	AudioEvent can be logged with timestamp, confidence, loudness, and duration.

Cameo implementation note

Model the detector output as an object flow typed by AudioEvent. Avoid modeling USB audio driver internals in the activity diagram; those details belong in the IBD and physical/software allocation views.

11. ACT-04 ClassifyAndLogBillieState

This activity is the bridge between raw perceptions and meaningful dog-state records. It is deliberately rule-based in the MVP. The action named Apply MVP rules should not call a reinforcement learning policy or Behavior AI module.

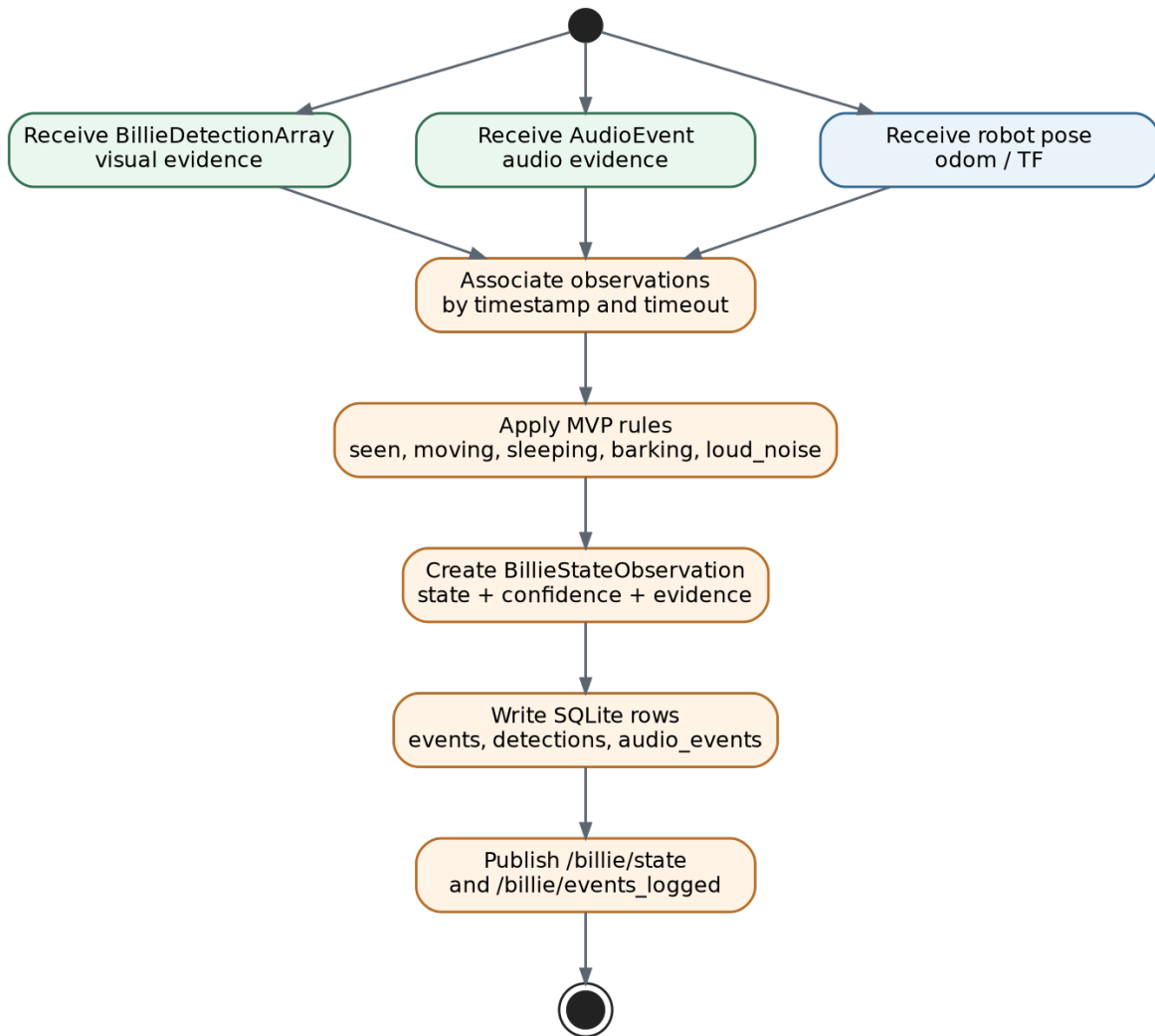


Figure 9. ACT-04 ClassifyAndLogBillieState: evidence fusion, state observation, and event logging.

Rule mapping to model as constraints or notes

Evidence condition	State label
Recent dog/Billie detection above confidence threshold	seen
Stable detection with low motion for configured duration	sleeping or resting
Detection position/bounding box changes above movement threshold	moving
Recent AudioEvent with event_type = bark	barking
Recent AudioEvent loud_noise with no bark confidence	loud_noise
No recent detection and no relevant audio event	not_seen or unknown depending on mission phase

Requirements satisfaction

ACT-04 directly satisfies REQ-MVP-006 and REQ-MVP-007. It also provides the data required by REQ-MVP-008 because the daily summary is generated from the SQLite events and observations produced here.

12. ACT-05 GenerateDailySummary

Daily summary generation should be modeled as deterministic aggregation from the event log. This keeps the MVP independent from cloud services and makes verification straightforward: seed the database, run the summary activity, inspect the output file and database row.

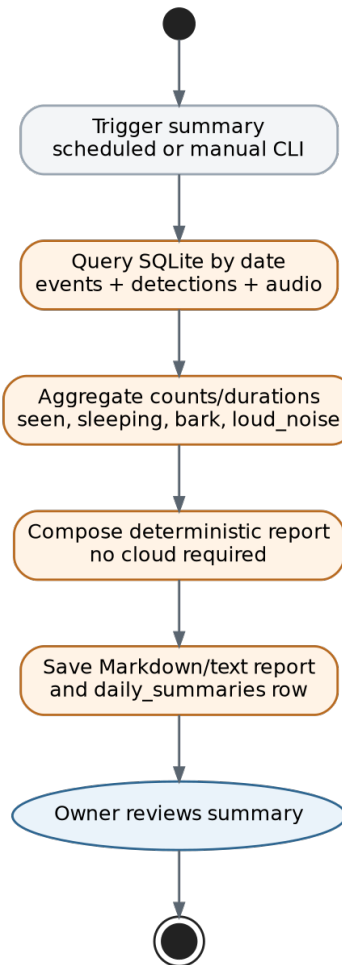


Figure 10. ACT-05 GenerateDailySummary: deterministic report generation from logged events.

Summary field	Source table / model object
Visual detection count	detections table and events with evidence_source = visual
Likely bark count	audio_events with event_type = bark
Loud noise count	audio_events with event_type = loud_noise
Resting/sleeping observations	events with state_label = sleeping/resting
Navigation/safety issues	events or diagnostics with mode = safe_stop, recovery, fault
Confidence notes	confidence fields from state, detection, and audio events

Requirements satisfaction

ACT-05 directly verifies and satisfies REQ-MVP-008. It also validates the upstream data contract for REQ-MVP-007 because summary generation cannot work unless logged observations are structured and queryable.

13. ACT-06 EnforceSafeMotion

The safety activity should be modeled separately because it cuts across teleoperation, navigation, search, and recovery. In the implementation, safety should sit between raw `/cmd_vel` and the motor controller as `/cmd_vel_safe`. In the model, this activity refines safety requirements and supports REQ-MVP-001 and REQ-MVP-009.

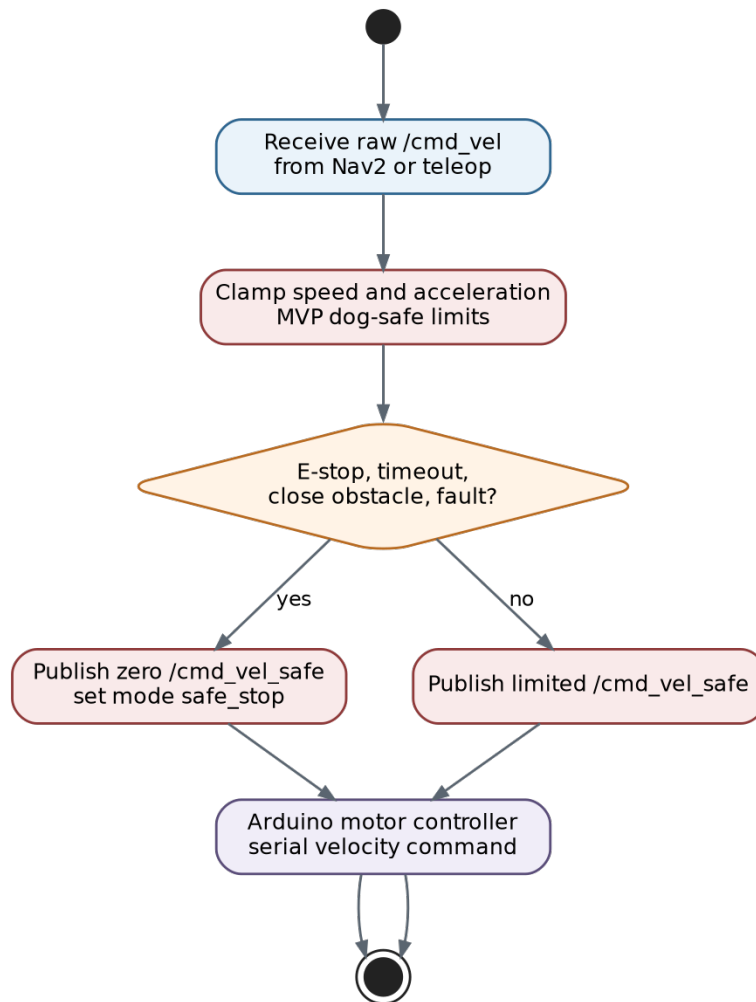


Figure 11. ACT-06 EnforceSafeMotion: velocity clamping, E-stop, watchdog, and safe motor command path.

Safety behavior	Modeling instruction
Velocity clamp	Represent as action with constraint values <code>max_linear_speed</code> and <code>max_angular_speed</code> .
E-stop	Model as decision input that forces zero velocity and <code>SystemMode = safe_stop</code> .
Timeout	Guard condition <code>[cmd_age > command_timeout]</code> routes to stop.
Fault	Sensor, battery, network, or navigation unrecoverable fault routes to <code>safe_stop</code> .
Motor output	Only <code>/cmd_vel_safe</code> reaches Arduino/motor controller in MVP architecture.

14. Allocation Model: Function to Software/Hardware

The allocation model is where the SysML architecture becomes implementable. Create allocation relationships from activities/functions to logical subsystems, then to software packages/nodes and physical components. Use allocation tables in Cameo so you can audit coverage quickly.

Activity/function	Logical block	Software node/package	Physical component
Navigate to waypoint	NavigationSubsystem	Nav2, billiebot_navigation	Jetson Orin Nano, RPLidar A1, wheel odometry
Build/localize map	NavigationSubsystem	SLAM Toolbox, AMCL config	Jetson, RPLidar, wheel encoders, optional IMU
Command mobile base	MobilitySubsystem + SafetySubsystem	billiebot_control, billiebot_safety	Jetson USB to Arduino Nano; motor driver; DC motors
Visual dog detection	VisualPerceptionSubsystem	billie_detector_node, DepthAI ROS	OAK-D Lite via USB
Audio event detection	AudioPerceptionSubsystem	audio_event_node	ReSpeaker XVF3800 via USB
State classification	StateEstimationSubsystem	billie_state_estimator	Jetson
Event logging	EventLoggingSubsystem	event_logger_node	Jetson NVMe / SQLite
Daily summary	DailySummarySubsystem	daily_summary CLI/node	Jetson; optional host review
Future policy decision	FutureBehaviorAISubsystem	reserved; no MVP node	future only

Cameo allocation construction

For each row above, create an allocate relationship from the action or activity to the logical block, and from the logical block or software node to the physical component. The allocation chain should allow a reviewer to start at REQ-MVP-004 and navigate to VisualPerceptionSubsystem, billie_detector_node, DepthAI/OAK-D Lite, and test_billie_visual_detection.py.

15. Traceability and Verification Model

The model should include a traceability pattern that connects requirements to activities, blocks, interfaces, test cases, and evidence. This makes the Cameo model useful during build and test rather than only during design.

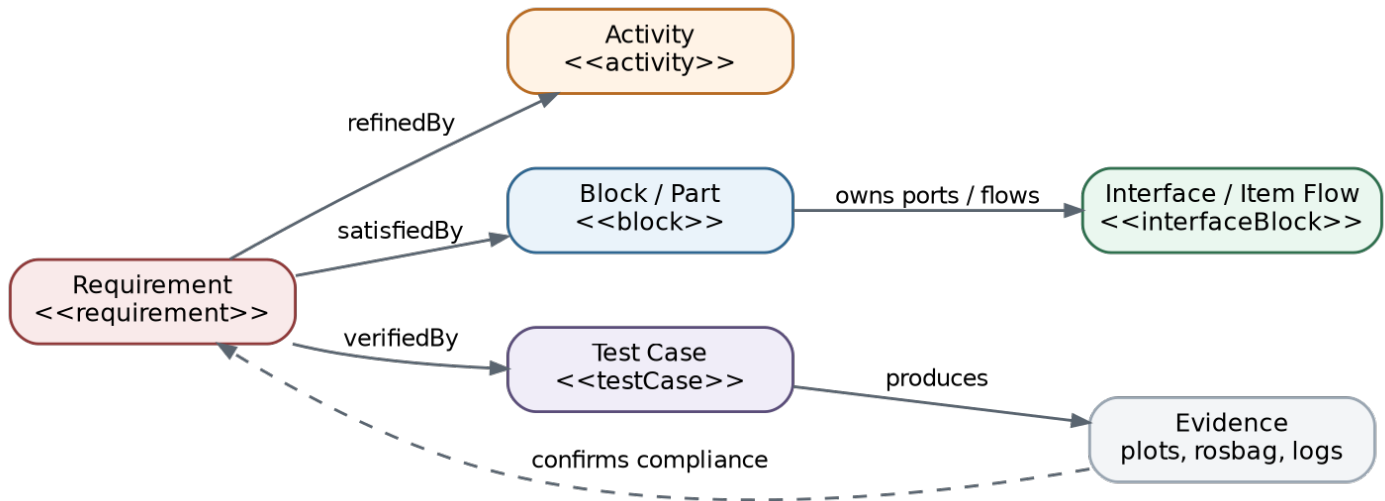


Figure 12. Recommended traceability pattern for requirements, design elements, tests, and evidence.

Requirements-to-activity satisfaction matrix

Requirement	ACT-01 Mission	ACT-02 Find Billie	ACT-03 Audio	ACT-04 State/Log	ACT-05 Summary	ACT-06 Safety
REQ-MVP-001 Navigate	Primary	Supports	-	-	-	Primary
REQ-MVP-002 Map/localize	Primary	Supports	-	-	-	-
REQ-MVP-003 Search	Primary	Primary	-	-	-	-
REQ-MVP-004 Visual detect	Invokes	Primary	-	Uses	-	-
REQ-MVP-005 Audio detect	Invokes	-	Primary	Uses	-	-
REQ-MVP-006 Classify state	Invokes	Provides evidence	Provides evidence	Primary	-	-
REQ-MVP-007 Log observations	Invokes	Provides evidence	Provides evidence	Primary	Uses	-
REQ-MVP-008 Daily summary	Invokes	-	-	Provides data	Primary	-
REQ-MVP-009 No engagement	Primary boundary	No action output	No action output	No policy call	-	Stops unsafe action
REQ-MVP-010 Future interfaces	References	Detection output usable later	Audio output usable later	State output usable later	Log usable later	Safety limits retained

Verification test-case mapping

Test case	Verifies	Evidence artifact to attach/link in Cameo
TC-001 Motor spin	Drive interface readiness for REQ-MVP-001	test log, video, command output
TC-004 Lidar scan	REQ-MVP-001, REQ-MVP-002	ros2 topic hz /scan, sample LaserScan
TC-010 SLAM map	REQ-MVP-002	map image, /map echo, saved map.yaml
TC-012 Nav2 waypoint	REQ-MVP-001, REQ-MVP-003	Nav2 action result, RViz screenshot, rosbag
TC-014 Visual detection	REQ-MVP-004, REQ-MVP-006	BillieDetection messages, debug image
TC-015 Bark detection	REQ-MVP-005, REQ-MVP-006	AudioEvent messages and timestamped test notes
TC-016 State logging	REQ-MVP-006, REQ-MVP-007	SQLite rows and topic output

Test case	Verifies	Evidence artifact to attach/link in Cameo
TC-017 Daily summary	REQ-MVP-008	summary Markdown/text and daily_summaries row
TC-019 Safety stop	REQ-SAFE series, REQ-MVP-001, REQ-MVP-009	clamped /cmd_vel_safe, safe_stop SystemMode

16. Future Behavior AI Integration - Modeled but Inactive

Future Behavior AI should be represented as a reserved architecture branch, not as part of the MVP mission behavior. This prevents scope creep while ensuring the MVP logs the observations needed for later learning. In Cameo, keep the future package separate and mark the blocks with notes such as "not executed in MVP".

Future block	Reserved responsibility	MVP status
EngagementManager	Decides whether engagement is allowed based on context and safety constraints.	Inactive/reserved
PolicyEngine	Selects observe, approach, retreat, speak, treat, or idle action.	Inactive/reserved
StressEstimator	Estimates stress from posture, motion, vocalization, avoidance, and owner feedback.	Inactive/reserved
RewardModel	Maps owner feedback and Billie response to reward signal.	Inactive/reserved
ContextualBanditLearner	Learns action preferences under context.	Inactive/reserved
InteractionActuatorSubsystem	Speaker, treat dispenser, lights, gestures, future actuators.	Inactive/reserved

Future interface stubs to model

```
EngagementContext
- time_of_day
- billie_state
- billie_location_context
- recent_bark_count
- recent_interaction_count
- owner_home_status
- robot_battery
- stress_score
- allowed_actions

EngagementAction
- action_type: observe | approach | retreat | speak | dispense_treat | idle
- intensity
- duration_s
- safety_constraints
- policy_confidence
```

Requirement satisfaction

This section satisfies REQ-MVP-010 by defining the future integration points. It also protects REQ-MVP-009 because none of these blocks are called by ACT-01 through ACT-06 in the MVP activity path.

17. Cameo Build Procedure

The following sequence gives a practical order for implementing the model in Cameo/MSOSA.

Build step	Instruction
Step 1 - Create project and packages	Create BillieBot_Model and packages 00 through 07. Add package diagrams only after creating the packages in the browser.
Step 2 - Create requirements	Enter MVP requirements REQ-MVP-001 through REQ-MVP-010. Add safety and future requirements in separate diagrams.
Step 3 - Create context blocks	Create BillieBot, Billie, Owner, Apartment, HostComputer, WiFiRouter_Opal, OptionalLLMCloud. Build BDD-01 System Context.
Step 4 - Create logical blocks	Create BillieBot subsystem blocks and composition associations. Build BDD-02 Logical Architecture.
Step 5 - Create interface blocks	Create LaserScanFlow, ImageDepthFlow, AudioEventFlow, VelocityCommandFlow, OdometryFlow, BillieDetectionFlow, StateObservationFlow, DailySummaryFlow.
Step 6 - Create IBD	Add subsystem parts, ports, and item flows to IBD-01 MVP Data and Control Flow.
Step 7 - Create activities	Build ACT-01 through ACT-06 with reusable call behavior actions and guard-labeled decision nodes.
Step 8 - Allocate functions	Allocate activities/actions to logical blocks, software nodes, and physical components.
Step 9 - Add verification	Create test cases TC-001 through TC-019 and verify relationships to requirements.
Step 10 - Generate matrices	Create requirements satisfaction matrix, activity allocation matrix, interface table, and verification matrix.
Step 11 - Add future package	Add reserved Behavior AI blocks and interfaces. Confirm no active MVP activity calls these blocks.

Recommended model review gates

Gate	Question to answer before proceeding
Requirements gate	Does every MVP requirement have at least one refining activity and satisfying block?
Behavior gate	Do ACT-01 through ACT-06 describe MVP behavior without Behavior AI scope creep?
Interface gate	Does every required software/hardware interface appear as a port/item flow or allocation?
Verification gate	Does every MVP requirement have at least one test case with defined evidence?
Future gate	Are future AI blocks present but inactive and uncalled in the MVP mission path?

18. Diagram Inventory to Create in Cameo

Use this as a checklist for the model browser. The model is not complete until these diagrams or equivalent matrices exist.

Diagram ID	Diagram name	Package	Purpose
PKG-01	Model Package Architecture	BillieBot_Model	Shows package organization and ownership.
BDD-01	System Context	00_Context	Shows system boundary and external actors/systems.
REQ-01	MVP Requirements	01_Requirements	Shows REQ-MVP hierarchy.
REQ-02	Safety Requirements	01_Requirements	Shows safety derived requirements.
REQ-03	Future Behavior AI Requirements	01_Requirements	Shows future reserved requirements separated from MVP.
BDD-02	Logical Architecture	03_Logical_Architecture	Shows BillieBot decomposition into logical subsystems.
IBD-01	MVP Data and Control Flow	03_Logical_Architecture	Shows ports, item flows, and ROS/data/control paths.
BDD-03	Physical Architecture	04_Physical_Architecture	Allocates logical subsystems to hardware/software components.
ACT-01	ExecuteMvpAutonomousMission	02_Functional_Architecture	Top-level MVP mission behavior.
ACT-02	FindBillie	02_Functional_Architecture	Waypoint search and visual detection behavior.
ACT-03	DetectAudioEvent	02_Functional_Architecture	Audio detection behavior.
ACT-04	ClassifyAndLogBillieState	02_Functional_Architecture	State fusion and event logging.
ACT-05	GenerateDailySummary	02_Functional_Architecture	Daily report generation.
ACT-06	EnforceSafeMotion	02_Functional_Architecture	Velocity clamp, E-stop, watchdog behavior.
MAT-01	Requirement Satisfaction Matrix	06_Verification	Requirement vs blocks and activities.
MAT-02	Verification Matrix	06_Verification	Requirement vs test case vs evidence.
MAT-03	Allocation Matrix	06_Verification or 04_Physical	Activity/function vs software/hardware allocation.

Appendix A - Suggested Safety and Future Requirements

Safety requirements

ID	Requirement text	Verification
REQ-SAFE-001	BillieBot shall limit maximum speed during apartment operation.	Inspect safety parameters and /cmd_vel_safe clamp behavior.
REQ-SAFE-002	BillieBot shall stop when collision risk exceeds threshold.	Obstacle-in-path stop test.
REQ-SAFE-003	BillieBot shall support manual emergency stop.	E-stop topic/service or physical kill switch test.
REQ-SAFE-004	BillieBot shall not intentionally contact Billie.	Dog-sized obstacle approach test; stop or retreat before contact.
REQ-SAFE-005	BillieBot shall fail safe on compute, network, or watchdog failure.	Kill ROS node or stop command stream and verify motor stop.
REQ-SAFE-006	BillieBot shall protect high-current power branches.	Physical inspection and load test.

Future requirements - not active in MVP

ID	Requirement text	MVP treatment
REQ-FUT-001	BillieBot shall support future engagement decision-making based on context.	Reserve interfaces only.
REQ-FUT-002	BillieBot shall support future learning from engagement outcomes.	Log context/state/action-ready observations, no learner active.
REQ-FUT-003	BillieBot shall support future stress-aware interaction constraints.	Reserve StressEstimator and safety ports.
REQ-FUT-004	BillieBot shall support future interaction actuators.	Reserve InteractionActuatorSubsystem, no MVP behavior path.

Appendix B - Starter Element List

Use these names as initial model elements. Rename only if you apply a consistent naming convention throughout the model.

Blocks

```
BillieBot
NavigationSubsystem
MobilitySubsystem
VisualPerceptionSubsystem
AudioPerceptionSubsystem
StateEstimationSubsystem
EventLoggingSubsystem
DailySummarySubsystem
SafetySubsystem
ComputeSubsystem
PowerSubsystem
CommunicationSubsystem
FutureBehaviorAISubsystem
RPLidarA1
OAKDLite
ReSpeakerXVF3800
BN0055_IMU
JetsonOrinNano
RaspberryPi5
ArduinoNano
DifferentialDriveBase
HostComputer
WiFiRouter_Opal
Apartment
Billie
Owner
```

Activities

```
ExecuteMvpAutonomousMission
FindBillie
NavigateToSearchWaypoint
PerformScanPattern
DetectVisualBillieCandidate
DetectAudioEvent
ClassifyAndLogBillieState
GenerateDailySummary
EnforceSafeMotion
ExecuteNavigationRecovery
EnterSafeIdle
```

Interface blocks / item flows

```
LaserScanFlow
ImageDepthFlow
AudioFrameFlow
ImuDataFlow
VelocityCommandFlow
SafeVelocityCommandFlow
OdometryFlow
TfFlow
BillieDetectionFlow
AudioEventFlow
BillieStateObservationFlow
EventLogStatusFlow
DailySummaryFlow
SystemModeFlow
EngagementContextFlow    # future only
EngagementActionFlow     # future only
```


Appendix C - Model Quality Checklist

Check	Pass criterion
Scope boundary	No MVP activity calls PolicyEngine, RewardModel, TreatDispenser, autonomous speech, or engagement action selection.
Requirements coverage	Every REQ-MVP requirement has at least one satisfy/refine relationship.
Test coverage	Every REQ-MVP requirement has at least one verify relationship to a test case.
Activity completeness	Each primary activity has initial/final nodes, decision guards, named actions, and typed object flows where appropriate.
Interface completeness	All ROS topics and major hardware signals appear in IBD item flows or interface tables.
Allocation completeness	Each function is allocated to logical block, software node/package, and physical component where applicable.
Safety	The model shows /cmd_vel_safe or equivalent safety-supervised motor command path.
Future readiness	Future Behavior AI package exists, but future blocks are marked inactive/reserved and not verified as MVP functionality.
Maintainability	Diagrams show views of reusable model elements, not free-floating duplicate text boxes.

End of report.